Picking up from the metrics deck

Your cheese detector earns **ROC AUC** 0.95 and a healthy PR AUC. The ranking is excellent. Ship it?

Not yet. AUC and PR rank batches *without* a threshold. To **act** — flag this batch or not — you must pick one cutoff c. And on a 3%-bad, asymmetric-cost line, the default $c = 0.5$ is almost never the right one.

This deck: how to turn a good *score* into a good *decision* — three ways to choose the operating point, the formula behind the best one, and the sklearn tool that does it for you.

Outline

The threshold is a knob

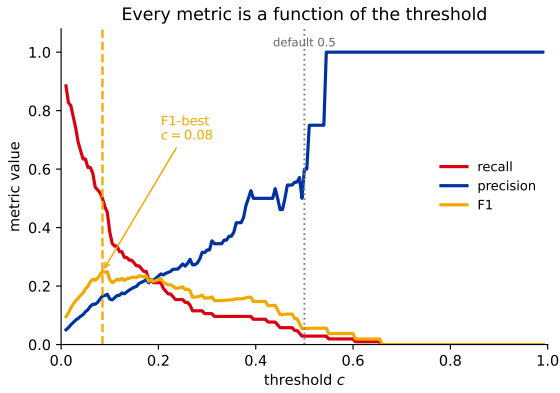
A classifier outputs a *score*; a cutoff c turns it into a yes/no. Every metric you care about moves as you slide c — so c is a **choice**, not a given.

Every metric is a function of the threshold

The model outputs scores; the **cutoff** c turns them into decisions. Sweep it:

- low $c \Rightarrow$ flag more $\Rightarrow$ recall up, precision down;
- high $c \Rightarrow$ flag fewer $\Rightarrow$ precision up, recall down.

The **default** 0.5 **is just one point**. Here the F1-best cutoff is ≈ 0.08 — the 3% imbalance pushes it far left of 0.5.



Three ways to pick the cutoff

Which rule you use depends on what you know: the **costs**, **nothing**, or a hard **requirement**. All three just slide the one number c.

Three ways to pick the cutoff

Choosing c is a **business** decision, not a modeling one. Which rule you use depends on what you know:

What you know	Pick the cutoff by...
the **costs** — a miss vs. a false alarm	**minimizing total cost** (cost-sensitive)
nothing — just balance the two errors	**Youden's J**: best catch-rate minus false-alarm-rate
a hard **requirement** ("catch $\geq$ 80%")	a **recall floor**: meet it, then cut false alarms

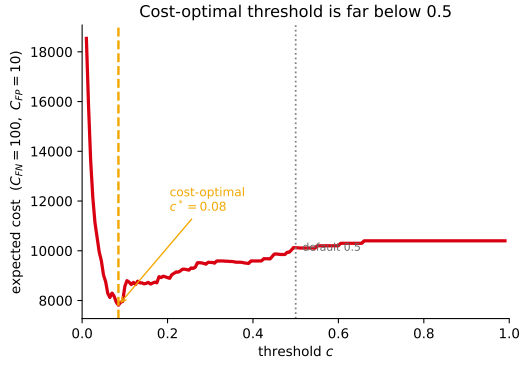
All three just slide the one number c — and every one of them beats blindly leaving it at 0.5. We take them in turn.

Cost-sensitive: when you know the price of a mistake

Each mistake has a price: a **miss (FN)** ships bad cheese; a **false alarm (FP)** wastes an inspection. Say a miss costs \$100, a false alarm \$10 — pick the cutoff with the **smallest total bill**:

$$c^* = \arg\min_c \; C_{FN}\, FN(c) + C_{FP}\, FP(c).$$

Sweep c, sum the cost, keep the cheapest: here $c^* \approx 0.08$, **far below** 0.5 (a miss hurts $10\times$ more).



Tune on a validation set, never the test set you report — else the cutoff is optimistically biased.

In sklearn: `TunedThresholdClassifierCV` tunes c by CV for any scorer (including a business-cost one) — code at the end of this section.

The cost-optimal cutoff has a closed form

Suppose the score is a **calibrated probability** $p = P(\text{bad} \mid \mathbf{x})$. Flagging is worth it only when the expected cost of flagging beats the expected cost of passing:

$$\underbrace{(1-p)\,C_{FP}}_{\text{flag: risk a false alarm}} \quad < \quad \underbrace{p\,C_{FN}}_{\text{pass: risk a miss}} \quad .$$

Solve for p:

$$p \;>\; \frac{C_{FP}}{C_{FP} + C_{FN}} \;=\; c^{*}.$$

Cheese factory ($C_{FN} = 100$, $C_{FP} = 10$):

$$c^{*} = \frac{10}{10 + 100} \approx 0.09.$$

Matches the swept cost curve (≈ 0.08) — and sits **far below** 0.5.

Big caveat: this formula assumes p is a *true probability*. Raw model scores are often *not* calibrated — then you fall back to sweeping c empirically (previous slide). Making scores trustworthy is the **next lecture: calibration**.

Worked example: sweep the cost by hand

Ten batches ranked by model score; **3 are truly bad**. Costs: a miss $C_{FN} = \$100$, a false alarm $C_{FP} = \$10$. Flag iff score $\geq c$; total bill $= 100\,FN + 10\,FP$.

batch	score	truth
b1	0.92	**bad**
b2	0.80	**bad**
b3	0.55	good
b4	0.40	**bad**
b5	0.30	good
b6	0.20	good
b7	0.12	good
b8–b10	≤ 0.08	good

c	FN	FP		**cost**
0.90	2	0		\$200
0.50	1	1		\$110
0.35	0	1	**\$10**	(cheapest)
0.15	0	3		\$30

The cheapest cutoff is $c = 0.35$, not 0.5. Leaving it at the default costs 11× the optimum here (\$110 vs \$10).

Wait — didn't the formula give $c^* \approx 0.09$? Yes, but that assumes p is a *calibrated* probability. These ten raw scores aren't (and the sample is tiny), so the discrete optimum sits higher. When scores aren't true probabilities you **sweep empirically**, exactly as here — and making them trustworthy is the **next lecture**.

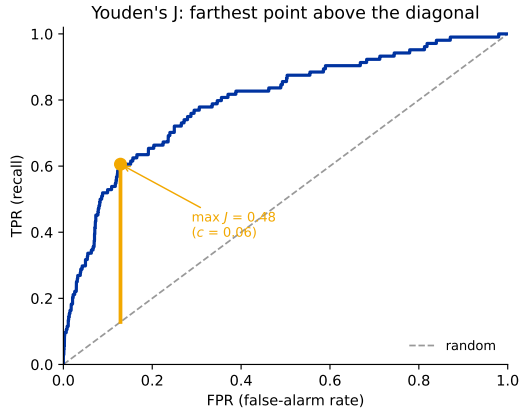
No costs? Youden's J balances the two error rates

No price tags, but you still want a sensible cutoff? **Youden's J** rewards catching bad batches and penalizes false alarms *equally*:

$$J = \underbrace{\mathrm{TPR}}_{\text{catch rate}} - \underbrace{\mathrm{FPR}}_{\text{false-alarm rate}} .$$

Pick the cutoff with the **biggest gap** between the two.

- On the ROC curve: the point **farthest above the diagonal**.
- Ignores how rare positives are, and ignores cost.
- A common default in medical screening.



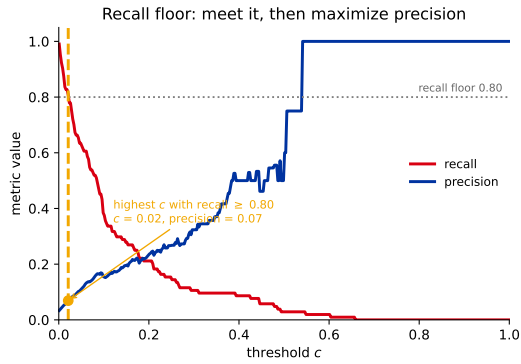
Watch out: J balances *rates*, not money — on rare-positive data it can land far from the cost-optimal cutoff. Use it only when you truly have no cost information.

A hard requirement? Use a recall floor

Often the business hands you a **rule**, not costs: *"we must catch at least* 80% *of bad batches."* So:

- Look at every cutoff that meets **recall** ≥ 0.80.
- Among those, take the **strictest** (highest c) — it meets the rule with the **fewest false alarms**.

Here that forces $c \approx 0.02$ and precision down to ~ 0.07: catching almost all of a rare class is *expensive* in false alarms.



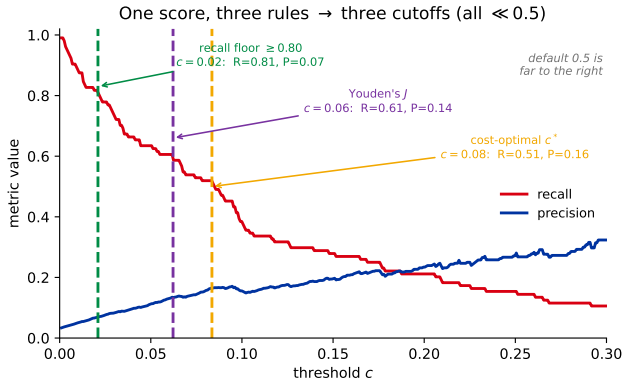
Mirror image: a **precision floor** ("at most X% wasted inspections") $\rightarrow$ maximize recall instead. In sklearn, `TunedThresholdClassifierCV` with a constrained scorer.

All three on one axis: one score, three cutoffs

Same detector, same scores — each rule lands on a **different** cutoff, and all three sit **far below the default** 0.5:

- **recall floor** lowest ($c \approx 0.02$): catches the most bad batches, and the most false alarms;
- **Youden's** J in the middle ($c \approx 0.06$);
- **cost-optimal** highest ($c \approx 0.08$): the fewest false alarms of the three.

The rarer the positive class, the further left *all* of them move — which is why 0.5 is almost never the right cutoff here.



One tool does all three: TunedThresholdClassifierCV

```python
from sklearn.model_selection import TunedThresholdClassifierCV
from sklearn.metrics import make_scorer

# business cost: a miss (FN) costs 100, a false alarm (FP) costs 10
def neg_cost(y_true, y_pred):
    fn = ((y_true == 1) & (y_pred == 0)).sum()
    fp = ((y_true == 0) & (y_pred == 1)).sum()
    return -(100 * fn + 10 * fp)          # higher is better

tuned = TunedThresholdClassifierCV(
    clf, scoring=make_scorer(neg_cost), cv=5).fit(X_train, y_train)

tuned.best_threshold_                     # the chosen cutoff (not 0.5)
tuned.predict(X_test)                     # applies that cutoff
```

Swap the rule by swapping the scorer: `scoring="balanced_accuracy"` $\approx$ Youden; a recall-floor scorer for a hard requirement. It tunes c **by CV on the training data** — so your reported test metrics stay honest. (sklearn $\geq$ 1.5.)

Recap: from score to decision

1. A classifier gives a **score**; a **cutoff** c turns it into a decision. Every metric moves with c — 0.5 **is just one arbitrary point**.
2. **Know the costs?** Minimize total cost. For calibrated p: $c^* = C_{FP}/(C_{FP} + C_{FN})$.
3. **Know nothing?** Youden's $J = \mathrm{TPR} - \mathrm{FPR}$ balances the two error rates.
4. **Have a hard rule?** A recall (or precision) floor: meet it, then minimize the other error.
5. **Always tune c on validation / CV**, never on the test set — `TunedThresholdClassifierCV` automates it.

One-line takeaway: the threshold is a business decision, not a modeling default — choose it on purpose. **Next:** calibration — making the score a *trustworthy probability* so c^* actually means what it says.